

HW2

February 5, 2026

Homework 2: Featurized Models for Sentiment Analysis

Shizhe Zhang

Due: February 10, 2026 @ 11:59pm

For this assignment, we provide an implementation of a simple binary classifier that will predict the sentiment of sentences from Yelp reviews based on a group of original features – provided by you!

Before diving into any code, please read through the associated instructions for an overview of the assignment and specific instructions on how to submit.

0.0.1 Assignment Overview and Learning Objectives

In this homework, you will 1. write code that generates features for the positive/negative sentiment classification of sentences from Yelp reviews 2. train a model on those features to predict the sentiment of new data.

During training, a basic logistic regression model (from the `scikit-learn` package) is trained with your created features to predict each review’s sentiment.

Your ultimate goal for this assignment is to **featurize the text creatively** and **optimize the accuracy** of the model on the test data.

Your main tasks are to implement:

1. A bag of words feature. Fill out the `bag_of_words` function in the notebook to represent a document through its bag of words. Represent each **lowercase** word tokenized with the NLTK `word_tokenize` function through its binary value. Remember that the feature value for a word that shows up in a review should be 1, no matter how many times it is mentioned. Please keep your code between the `#BEGIN/#END SOLUTION` flags.
2. Create **three** different classes of features. Implement them in the `feature1`, `feature2`, and `feature3` functions. Create features that you think would perform better than bag of words, and assess their independent performance on the development data. Describe your reasoning and include accuracy scores on the `dev.txt` data (printed in the Colab output). **You must report your reasoning and development scores in the table provided.**

It is not required that your features actually perform well, but your justification and reasoning should be defensible. We are looking for thoughtful insights as to why your original features should out-perform the simple bag of words implementation.

For more feature ideas consult [SLP Ch. 4](#) and lecture slides, and please do keep the following constraints in mind: - You should only change the code for `bag_of_words`, `feature1`, `feature2`, `feature3` and `combiner_function` along with any code you need to support those functions. You must use the classifier provided. - You are free to use external dictionaries (such as LIWC or AFINN); if you use these, upload them to [Gradescope](#) as well with your `HW2.ipynb`. - Do not use pre-trained word vectors (including static embeddings like word2vec or contextual embeddings like BERT). - Do not use the predictions of any other supervised model or LLM on this data as a feature for logistic regression (e.g., do not train a separate CNN on this data, make predictions on the dev data using the CNN, and then treat your CNN prediction as a feature). - Do not alter `{train,dev,test}.txt` or find additional sources of training data. This homework is focusing on how you represent your data, not other aspects of the training regime. - Do not import any other additional libraries.

At the end of the notebook we've included several other methods that you can use to interrogate your model; please feel free to use these to assess what kind of mistakes your current model is making in order to help brainstorm new features.

Once you've defined your features, described them, and evaluated their performance on the development data, it's time to make predictions on the test data. Adapt the `combiner_function` cell to include whichever features you like and execute it to make predictions on the test data. This will create the file `combiner_function_predictions.csv`; download this from Colab (using e.g. the file manager on the left panel) and submit it to [Gradescope](#), along with your `HW2.ipynb` and a pdf of the notebook.

```
[2]: import sys, argparse
      from scipy import sparse
      from sklearn import linear_model
      from collections import Counter
      import numpy as np
      from sklearn.utils._testing import ignore_warnings
      from sklearn.exceptions import ConvergenceWarning
      import re
      from collections import Counter, defaultdict
      import operator
      import nltk
      import csv
      from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
      import matplotlib.pyplot as plt
      import pandas as pd
      from pandas import option_context
```

```
[3]: !python -m nltk.downloader punkt_tab
```

```
<frozen runpy>:128: RuntimeWarning: 'nltk.downloader' found in sys.modules after
import of package 'nltk', but prior to execution of 'nltk.downloader'; this may
result in unpredictable behaviour
```

```
[nltk_data] Downloading package punkt_tab to
```

```
[nltk_data]      /Users/zhangshizhe/nltk_data...
```

```
[nltk_data]   Package punkt_tab is already up-to-date!
```

0.0.2 Intro: Gather Data + Create Classifier

Gather Data Let's download the data we'll use for training and development, and also the data we'll make predictions with.

```
[4]: # Get data
!wget https://github.com/dbamman/nlp-course/raw/refs/heads/main/HW/data/train.
    ↪txt -O train.txt
!wget https://github.com/dbamman/nlp-course/raw/refs/heads/main/HW/data/test.
    ↪txt -O test.txt
!wget https://github.com/dbamman/nlp-course/raw/refs/heads/main/HW/data/dev.txt
    ↪-O dev.txt
```

```
--2026-02-05 11:42:38-- https://github.com/dbamman/nlp-
course/raw/refs/heads/main/HW/data/train.txt
Resolving github.com (github.com)... 140.82.114.4
Connecting to github.com (github.com)|140.82.114.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/train.txt [following]
--2026-02-05 11:42:39-- https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/train.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
2606:50c0:8002::154, 2606:50c0:8003::154, 2606:50c0:8000::154, ...
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|2606:50c0:8002::154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3621775 (3.5M) [text/plain]
Saving to: 'train.txt'
```

```
train.txt          100%[=====>]    3.45M  17.0MB/s   in 0.2s
```

```
2026-02-05 11:42:39 (17.0 MB/s) - 'train.txt' saved [3621775/3621775]
```

```
--2026-02-05 11:42:40-- https://github.com/dbamman/nlp-
course/raw/refs/heads/main/HW/data/test.txt
Resolving github.com (github.com)... 140.82.114.4
Connecting to github.com (github.com)|140.82.114.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/test.txt [following]
--2026-02-05 11:42:40-- https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/test.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
2606:50c0:8002::154, 2606:50c0:8003::154, 2606:50c0:8000::154, ...
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|2606:50c0:8002::154|:443... connected.
HTTP request sent, awaiting response... 200 OK
```

Length: 207117 (202K) [text/plain]
Saving to: 'test.txt'

test.txt 100%[=====>] 202.26K --.-KB/s in 0.04s

2026-02-05 11:42:40 (4.90 MB/s) - 'test.txt' saved [207117/207117]

```
--2026-02-05 11:42:41-- https://github.com/dbamman/nlp-
course/raw/refs/heads/main/HW/data/dev.txt
Resolving github.com (github.com)... 140.82.114.4
Connecting to github.com (github.com)|140.82.114.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/dev.txt [following]
--2026-02-05 11:42:41-- https://raw.githubusercontent.com/dbamman/nlp-
course/refs/heads/main/HW/data/dev.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)...
2606:50c0:8002::154, 2606:50c0:8003::154, 2606:50c0:8000::154, ...
Connecting to raw.githubusercontent.com
(raw.githubusercontent.com)|2606:50c0:8002::154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 214630 (210K) [text/plain]
Saving to: 'dev.txt'
```

dev.txt 100%[=====>] 209.60K --.-KB/s in 0.04s

2026-02-05 11:42:41 (5.69 MB/s) - 'dev.txt' saved [214630/214630]

```
[5]: trainingFile = "train.txt"
evaluationFile = "dev.txt"
testFile = "test.txt"
```

Define Classifier class Next, we've created a Binary Classifier. This class will let us learn the traits associated with positive and negatively classed review sentences in order to make predictions on our test set.

```
[6]: #####
## Do not edit this block of code.
## This defines the classification class which
## loads the data and sets up the model.
#####

class Classifier:

    def __init__(self, feature_method, L2_regularization_strength=1.0,
min_feature_count=1):
```

```

        self.feature_vocab = {}
        self.feature_method = feature_method
        self.log_reg = None
        self.L2_regularization_strength=L2_regularization_strength
        self.min_feature_count=min_feature_count

        self.trainX, self.trainY, self.trainOrig = self.process(trainingFile,
↪training=True)
        self.devX, self.devY, self.devOrig = self.process(evaluationFile,
↪training=False)
        self.testX, _, self.testOrig = self.process(testFile, training=False)

# Read data from file
def load_data(self, filename):
    data = []
    with open(filename, encoding="utf8") as file:
        reader = csv.reader(file, delimiter='\t')
        for row in reader:
            idd = row[0]
            label = row[1]
            text = row[2]

            data.append((idd, label, text))

    return data

# Featurize entire dataset
def featurize(self, data):
    featurized_data = []
    for idd, label, text in data:
        feats = self.feature_method(text)
        featurized_data.append((label, feats))
    return featurized_data

# Read dataset and returned featurized representation as sparse matrix +
↪label array
def process(self, dataFile, training = False):
    original_data = self.load_data(dataFile)
    data = self.featurize(original_data)

    if training:
        fid = 0
        feature_doc_count = Counter()
        for label, feats in data:
            for feat in feats:
                feature_doc_count[feat] += 1

```

```

        for feat in feature_doc_count:
            if feature_doc_count[feat] >= self.min_feature_count:
                self.feature_vocab[feat] = fid
                fid += 1

F = len(self.feature_vocab)
D = len(data)
X = sparse.dok_matrix((D, F))
Y = [None]*D
for idx, (label, feats) in enumerate(data):
    for feat in feats:
        if feat in self.feature_vocab:
            X[idx, self.feature_vocab[feat]] = feats[feat]
    Y[idx] = label

return X, Y, original_data

def load_test(self, dataFile):
    data = self.load_data(dataFile)
    data = self.featurize(data)

    F = len(self.feature_vocab)
    D = len(data)
    X = sparse.dok_matrix((D, F))
    Y = [None]*D
    for idx, (data_id, feats) in enumerate(data):
        for feat in feats:
            if feat in self.feature_vocab:
                X[idx, self.feature_vocab[feat]] = feats[feat]
        Y[idx] = data_id

    return X, Y

# Train model and evaluate on held-out data
@ignore_warnings(category=ConvergenceWarning)
def evaluate(self):
    (D,F) = self.trainX.shape
    self.log_reg = linear_model.LogisticRegression(C = self.
↪L2_regularization_strength, max_iter=1000)
    self.log_reg.fit(self.trainX, self.trainY)
    training_accuracy = self.log_reg.score(self.trainX, self.trainY)
    development_accuracy = self.log_reg.score(self.devX, self.devY)
    print("Method: %s, Features: %s, Train accuracy: %.3f, Dev accuracy: %.
↪3f" % (self.feature_method.__name__, F, training_accuracy,
↪development_accuracy))

```

```

# Predict labels for new data
def predict(self):
    predX = self.log_reg.predict(self.testX)

    with open("%s_%s" % (self.feature_method.__name__, "predictions.csv"),
↪ "w", encoding="utf8") as out:
        writer=csv.writer(out)
        writer.writerow(["Id", "Expected"])
        for idx, data_id in enumerate(self.testX):
            writer.writerow([self.testOrig[idx][0], predX[idx]])
    out.close()

def printWeights(self, n=10):

    reverse_vocab=[None]*len(self.log_reg.coef_[0])
    for k in self.feature_vocab:
        reverse_vocab[self.feature_vocab[k]]=k

    # binary
    if len(self.log_reg.classes_) == 2:
        weights=self.log_reg.coef_[0]

        cat=self.log_reg.classes_[1]
        for feature, weight in list(reversed(sorted(zip(reverse_vocab,
↪ weights), key = operator.itemgetter(1))))[:n]:
            print("%s\t%.3f\t%s" % (cat, weight, feature))
        print()

        cat=self.log_reg.classes_[0]
        for feature, weight in list(sorted(zip(reverse_vocab, weights),
↪ key = operator.itemgetter(1))))[:n]:
            print("%s\t%.3f\t%s" % (cat, weight, feature))
        print()

    # multiclass
    else:
        for i, cat in enumerate(self.log_reg.classes_):

            weights=self.log_reg.coef_[i]

            for feature, weight in list(reversed(sorted(zip(reverse_vocab,
↪ weights), key = operator.itemgetter(1))))[:n]:
                print("%s\t%.3f\t%s" % (cat, weight, feature))
            print()

```

Simple Classifier example Below is an example of select sentences with labels: - **Positive** (pos): My last mani lasted almost a month with no chips. - **Negative** (neg): We went tonight and are now kicking ourselves.

Let's create an initial classifier based on a really simple feature using a dictionary:

- if the abstract contains the words “love” or “like”, the `contains_positive_word` feature will fire, and
- if it contains either “hate” or “dislike”, the `contains_negative_word` will fire.

Note how we use `nltk.word_tokenize` to tokenize the text into its discrete words (the documentation for which can be found [here](#)).

```
[7]: def simple_featurize(text):
    feats = {}
    words = nltk.word_tokenize(text)

    for word in words:
        word=word.lower()
        if word == "love" or word == "like":
            feats["contains_positive_word"] = 1
        if word == "hate" or word == "dislike":
            feats["contains_negative_word"] = 1

    return feats
```

Now let's see how that feature performs on the development data.

Note the `L2_regularization_strength` parameter specifies the strength of the L2 regularizer (values closer to 0 = stronger regularization), and `min_feature_count` specifies how many data points need to contain a feature for it to be passed into the model as a feature. Both are ways to prevent the model from overfitting and achieve higher performance.

```
[8]: simple_classifier = Classifier(simple_featurize, L2_regularization_strength=1.
    ↪0, min_feature_count=1)
simple_classifier.evaluate()
```

Method: simple_featurize, Features: 2, Train accuracy: 0.596, Dev accuracy: 0.501

So we've created a classifier. But is its accuracy score any good? Let's calculate the accuracy of a “majority classifier” to provide some context. This determines the most-represented (majority) class in the training data, and then predicts every test point to be this class.

```
[9]: def majority_class(trainY, devY):
    labelCounts=Counter()
    for label in trainY:
        labelCounts[label]+=1
    majority_class=labelCounts.most_common(1)[0][0]

    correct=0.
```



```

for label in devY:
    if label == majority_class:
        correct+=1

print("Majority class: %s\tDev accuracy: %.3f" % (majority_class, correct/
↪len(devY)))
majority_class(simple_classifier.trainY, simple_classifier.devY)

```

Majority class: pos Dev accuracy: 0.500

The feature we created in `simple_featurize`, evidently, doesn't have a whole lot of legs. In the next portion of the homework, you'll be designing a few features of your own in the hopes of achieving the highest accuracy possible.

0.1 Deliverable 1

Your job in this homework is to implement a binary bag-of-words model (i.e., one that assigns a feature value of 1 to each word type that is present in the text); and to brainstorm **3 additional** distinct classes of features, justify why they might help improve the performance *over a bag of words* for this task, implement them in code, and then assess their independent performance on the development data.

Describe your new features, one paragraph (around 50 words) each:

Bag of words (Example): The bag-of-words feature converts review text into binary indicators that signal the presence or absence of each word, without considering the order. This approach simplifies text representation, enabling the classifier to focus on distinguishing words for each review.

Feature 1: Identify and count the presence of strong sentiment words using a predefined lexicon (e.g., 'un'). This feature captures the negative intensity of the review, which is often a strong indicator of sentiment. But it also includes words like 'understand' which may not contribute to sentiment, so careful selection of the lexicon is crucial.

Feature 2: Measure the length of the review in terms of word count. Longer reviews may provide more context and detail, which could be a sign of more nuanced and positive sentiment. And the average length of words indicates the complexity of the language used. If a review uses longer words, it may suggest a more thoughtful and positive sentiment.

Feature 3: Count the number of exclamation marks and question marks in the review. Exclamation marks often indicate strong emotions, which can be associated with positive or negative sentiment. Question marks may indicate uncertainty or dissatisfaction, which could correlate with negative sentiment. These punctuation features can provide additional context to the sentiment expressed in the text.

Now, implement the features in the specified `bag_of_words`, `feature1`, `feature2`, and `feature3` functions, and execute each respective classifier to show its performance.

Note that it is not required for your features to actually perform well, but your justification for why it *should* perform better than a bag of words should be defensible. Consider the type of data you are working with: what do you look for when writing/reading a review?

0.1.1 Implement Bag-of-Words

```
[10]: def bag_of_words(text):
    # Here the `feats` dict should contain the features -- the key should be
    ↪ the feature name,
    # and the value is the feature value. See `simple_featurize` for an
    ↪ example.

    """YOUR CODE HERE"""
    feats = {}
    words = nltk.word_tokenize(text)
    # BEGIN SOLUTION
    words = set(words)
    for word in words:
        word = word.lower()
        feats[word] = 1

    return feats
    # END SOLUTION
```

```
[11]: #####
    ## Do not edit this block of code, except for the L2_regularization_strength
    ↪ and min_feature_count parameters
    #####

    bow_classifier = Classifier(bag_of_words, L2_regularization_strength=1.0,
    ↪ min_feature_count=1)
    bow_classifier.evaluate()
```

Method: bag_of_words, Features: 22970, Train accuracy: 0.848, Dev accuracy: 0.733

If you correctly implemented `bag_of_words`, you may expect a train accuracy of ~0.849 and a dev accuracy of ~0.732.

0.1.2 Implement Original Features

```
[47]: def feature1(text):
    # Here the `feats` dict should contain the features -- the key should be
    ↪ the feature name,
    # and the value is the feature value. See `simple_featurize` for an
    ↪ example.

    """YOUR CODE HERE"""
    # BEGIN SOLUTION
    feats = {}
    feats["contains_negative_words"] = 0
    words = nltk.word_tokenize(text)
```

```

for word in words:
    word = word.lower()
    if word.startswith(('un', 'dis')):
        feats["contains_negative_words"] += 1
return feats
# END SOLUTION

```

```

[48]: #####
## Do not edit this block of code, except for the L2_regularization_strength,
    ↪ and min_feature_count parameters
#####

classifier1 = Classifier(feature1, L2_regularization_strength=1.0,
    ↪ min_feature_count=1)
classifier1.evaluate()

```

Method: feature1, Features: 1, Train accuracy: 0.602, Dev accuracy: 0.522

```

[18]: def feature2(text):
    # Here the `feats` dict should contain the features -- the key should be
    ↪ the feature name,
    # and the value is the feature value. See `simple_featurize` for an
    ↪ example.

    """YOUR CODE HERE"""
    # BEGIN SOLUTION
    feats = {}
    words = nltk.word_tokenize(text)
    feats["length_of_comments"] = len(words)
    feats["avg_length_of_words"] = sum(map(len, words)) / len(words)

    return feats
    # END SOLUTION

```

```

[19]: #####
## Do not edit this block of code, except for the L2_regularization_strength,
    ↪ and min_feature_count parameters
#####

classifier2 = Classifier(feature2, L2_regularization_strength=1.0,
    ↪ min_feature_count=1)
classifier2.evaluate()

```

Method: feature2, Features: 2, Train accuracy: 0.594, Dev accuracy: 0.505

```
[20]: def feature3(text):
    # Here the `feats` dict should contain the features -- the key should be
    ↪ the feature name,
    # and the value is the feature value. See `simple_featurize` for an
    ↪ example.

    """YOUR CODE HERE"""
    # BEGIN SOLUTION
    feats = {}
    feats["exclamations"] = 0
    feats["questions"] = 0
    words = nltk.word_tokenize(text)

    for word in words:
        if word == '!':
            feats["exclamations"] += 1
        if word == '?':
            feats["questions"] += 1
    return feats
    # END SOLUTION
```

```
[21]: #####
## Do not edit this block of code, except for the L2_regularization_strength
    ↪ and min_feature_count parameters
#####

classifier3 = Classifier(feature3, L2_regularization_strength=1.0,
    ↪ min_feature_count=1)
classifier3.evaluate()
```

Method: feature3, Features: 2, Train accuracy: 0.597, Dev accuracy: 0.507

0.2 Deliverable 2

The two cells in “Combine your features” will generate a file named `combiner_function_predictions.csv`.

Download this file (using the file manager on the left panel in Colab) and submit this to Gradescope along with your notebook.

Please do not change the auto-generated filename!

0.2.1 Combine your features

Next, let’s combine any or all the features you have developed into one big model and make predictions on the test data. There is no exact number/threshold we’re looking for, accuracy-wise, but the combiner function should *generally* have a higher accuracy than BoW on its own (assuming your features are adding additional information beyond what BoW is adding).

You don't need to edit the following cell, unless you want to change which features are handed off to the "combined" model.

```
[68]: def combiner_function(text):

    # Here the `all_feats` dict should contain the features -- the key should
    ↪ be the feature name,
    # and the value is the feature value. See `simple_featurize` for an
    ↪ example.
    # at the moment, all 4 of: bag of words and your 3 original features are
    ↪ handed off to the combined model
    # update the values within [bag_of_words, feature1, feature2, feature3] to
    ↪ change this.

    """YOUR CODE HERE"""
    all_feats={}
    # BEGIN SOLUTION
    words = nltk.word_tokenize(text)
    # BEGIN SOLUTION
    all_feats['length_of_comments'] = len(words)

    for word in words:
        word = word.lower()
        all_feats[word] = 1
        if word == '!':
            all_feats["exclamation"] = all_feats.get("exclamation",0) + 1
        elif word == '?':
            all_feats["questions"] = all_feats.get("questions",0) + 1
        elif word.startswith(('un','dis')):
            all_feats["contains_nagetive_words"] = all_feats.
    ↪ get("contains_nagetive_words",0) + 1
            # if word == 'but':
            #     all_feats.clear()

    return all_feats
    # END SOLUTION
```

```
[69]: #####
    ## Do not edit this block of code, except for the L2_regularization_strength
    ↪ and min_feature_count parameters
    #####

    big_classifier = Classifier(combiner_function, L2_regularization_strength=5,
    ↪ min_feature_count=1)
    big_classifier.evaluate()

    #generate csv file with prediction output on test data
```

```
big_classifier.predict()
```

Method: combiner_function, Features: 22972, Train accuracy: 0.890, Dev accuracy: 0.731

We recommend that you download `combiner_function_predictions.csv` now in case your run-time gets disconnected.

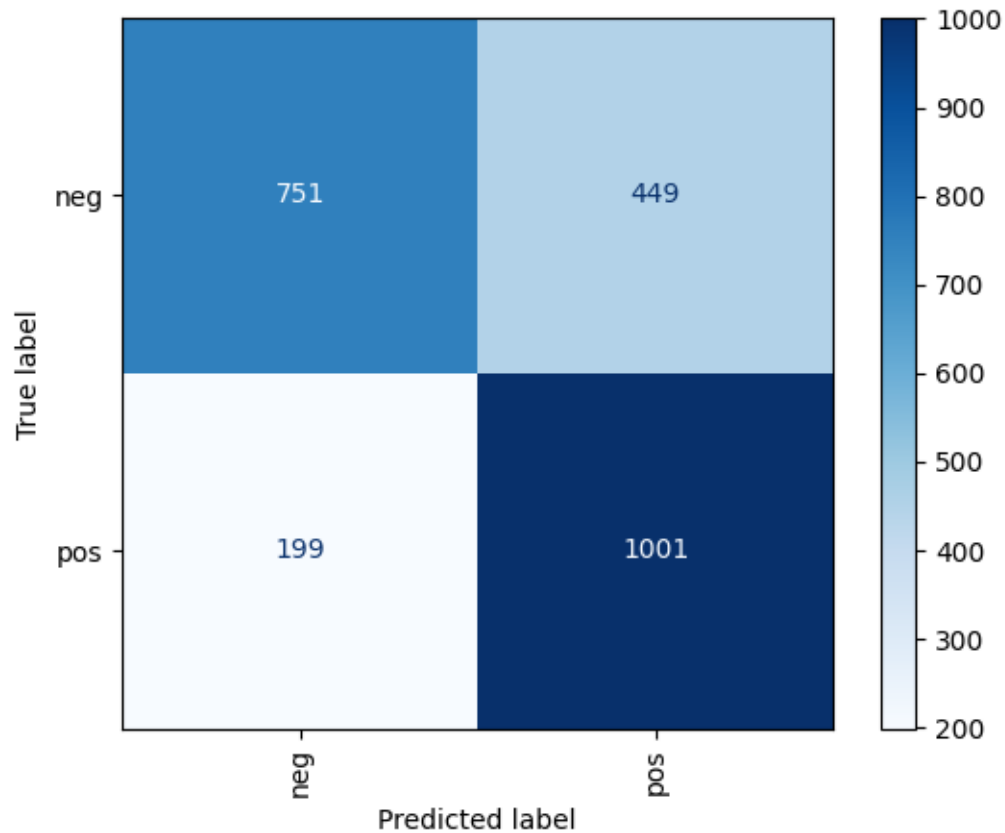
0.3 Further Exploration: Interrogating classifiers

Note: No deliverables are in this section; it's optional. Treat this portion as a useful tool for further understanding the features you worked on and how they affect the classifier.

Below you will find several ways in which you can interrogate your model to get ideas on ways to improve its performance.

1. First, let's look at the confusion matrix of its predictions (where we can compare the true labels with the predicted labels). What kinds of mistakes is it making? (While this is mainly helpful in the context of multiclass classification, we can still see if there's a bias toward predicting a specific class in the binary setting as well).

```
[51]: def print_confusion(classifier):  
    cm = confusion_matrix(classifier.devY, classifier.log_reg.  
        ↪predict(classifier.devX))  
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,  
        ↪display_labels=classifier.log_reg.classes_)  
    disp.plot(xticks_rotation="vertical", values_format="d", cmap='Blues')  
    plt.show()  
  
print_confusion(big_classifier)
```



- Next, let's look at the features that are most important for each of the classes (ranked by how strong their corresponding coefficient is). Do the features you are defining help in the ways you think they should? Do sets of successful features suggests others, or complementary features that may provide a different view on the data?

```
[57]: big_classifier.printWeights(n=25)
```

```
pos      3.609   crown
pos      3.598  discounted
pos      3.542    gem
pos      3.498   nook
pos      3.388  bearable
pos      3.387  performing
pos      3.361  principal
pos      3.337   earn
pos      3.326  relieved
pos      3.293  turnover
pos      3.283   10am
pos      3.268  fortunate
pos      3.236  genuinely
pos      3.234   reads
```

pos	3.174	butcher
pos	3.174	thankful
pos	3.158	devoured
pos	3.156	mastro
pos	3.150	student
pos	3.132	roots
pos	3.117	bomb
pos	3.112	refreshing
pos	3.112	hollandaise
pos	3.087	yay
pos	3.071	joey
neg	-3.968	neglected
neg	-3.785	sarcastic
neg	-3.765	struggle
neg	-3.635	thieves
neg	-3.628	brutal
neg	-3.619	sommelier
neg	-3.582	off-site
neg	-3.543	drawback
neg	-3.501	scary
neg	-3.486	sarcasm
neg	-3.486	admitted
neg	-3.474	dumb
neg	-3.434	spotty
neg	-3.387	home.
neg	-3.386	nastiest
neg	-3.370	subpar
neg	-3.350	over-looked
neg	-3.308	staffing
neg	-3.305	sonora
neg	-3.277	gripe
neg	-3.267	riddens
neg	-3.262	ankle
neg	-3.256	hazelnut
neg	-3.238	midway
neg	-3.220	drive-through

- Next, let's look at the individual data points that are the hardest to classify correctly. Does it suggest any features you might create to disentangle them?

```
[78]: def analyze(classifier):

    probs=classifier.log_reg.predict_proba(classifier.devX)
    predicts=classifier.log_reg.predict(classifier.devX)

    classes={}
```



```

for idx, lab in enumerate(classifier.log_reg.classes_):
    classes[lab]=idx

mistakes={}
for i in range(len(probs)):
    if predicts[i] != classifier.devY[i]:
        predicted_lab_idx=classes[predicts[i]]
        mistakes[i]=probs[i][predicted_lab_idx]

frame=[]
sorted_x = sorted(mistakes.items(), key=operator.itemgetter(1),
reverse=True)
for k, v in sorted_x:
    idd=classifier.devOrig[k][0]
    text=classifier.devOrig[k][2]
    frame.append([idd, v, classifier.devY[k], predicts[k], text])

df=pd.DataFrame(frame, columns=["id", "P(predicted class confidence)",
"Human label", "Prediction", "Text"])

with option_context('display.max_colwidth', 80):
    display(df.head(n=20))

```

```
[79]: analyze(big_classifier)
```

	id	P(predicted class confidence)	Human label	Prediction	\
0	40109	0.998946	neg	pos	
1	41483	0.998926	neg	pos	
2	41098	0.998795	pos	neg	
3	40495	0.998089	neg	pos	
4	41858	0.998070	neg	pos	
5	42068	0.996314	neg	pos	
6	41571	0.995551	pos	neg	
7	41403	0.994211	neg	pos	
8	42167	0.993702	neg	pos	
9	41931	0.993608	neg	pos	
10	40712	0.993418	neg	pos	
11	41242	0.993197	neg	pos	
12	41826	0.992594	pos	neg	
13	41012	0.990454	neg	pos	
14	41552	0.990059	neg	pos	
15	41804	0.988454	neg	pos	
16	41078	0.986838	neg	pos	
17	41908	0.983766	neg	pos	
18	41668	0.982885	neg	pos	
19	41239	0.982498	neg	pos	

↩Text

0 If you live there, you will have to do their job when it comes to maintenanc...
1 He told me to give him a break, but as a customer I told him that it was his...
2 I must commend the management...a family of DB's came in with their little ...
3 But similar to other reviews, they pushed me right into the payment process ...
4 Philly cheese steak was filled with some kind of ground-up meat, mixed with ...
5 She was more than happy to oblige AND NEVER mentioned that it would cost us ...
6 We watched a young waitress struggle with the outside heaters for a while an...
7 While waiting Katie had Jimmy Johns delivered and ate her submarine sandwich...
8 By the time his replacement steak was brought out sans hair, my plate was cl...
9 "Kobe Sliders" at your local bar and grill for \$12 have more in common with ...
10 And the manager in the morning who was equally curt and offered to waive the...
11 I mean Google the thing for F sake and you will see many amazing ways of pre...
12 Yet it only takes few minutes to get thru the line at Dutch bros because the...
13 The gentrification of BBQ continues in N. Scottsdale with Naked

↩BBQ.

14 I was also expecting the Croque Madame to be bigger than it actually was & l...
15 I spent the entire evening squished into a space where my butt was forward a...
16 (the twin pullout is cheap and you can feel the springs so have fun sleeping...
17 I had laser treatments on my face and those required numbing cream before ea...
18 Why they didn't ask for it prior to this point, knowing the repair they were...
19 Good riddance Justin Goodman and majestic

↩prop.

0.3.1 Closing and Submission

Congratulations on finishing HW2!

For this homework, you will be able to see a leaderboard on Gradescope. This is **optional** (for fun) and to see how your featurized model may compare to your classmates!
You will still be graded on good effort, not on an autograder criteria.

There are 3 deliverables, each to be submitted on [Gradescope](#):

1. HW 2 Test Set Predictions

- Submit the file `combiner_function_predictions.csv`. You can find files associated with the notebook if you click on the folder icon on the left-side panel of your Colab notebook. *Don't alter the file name.*
- Folder icon (left panel) → download `combiner_function_predictions.csv`

2. HW 2 Code

- Download your Colab notebook as an `.ipynb` file (File → Download → Download `.ipynb`)
- Submit `HW2.ipynb`. *Don't alter the file name.*
- If you used any external dictionaries, please attach them as well.

3. HW 2 PDF

- If using Colab, you can use the cell below to generate a pdf.

Please ensure that you submit the predictions, completed notebook (notebook and pdf), and any external dictionaries used onto [Gradescope](#) before February 10 at 11:59pm. The notebook you upload to [Gradescope](#) must be named HW2.ipynb.

```
[ ]: #EXPORT_EXCLUDE#

#@markdown This is a helper function to generate a PDF in Colab.
#@markdown If you are using Jupyter notebook, you can do `File > Save and
↳Export Notebook as HTML`, then save the resulting HTML file as a PDF.
#@markdown Alternatively, in Jupyter notebook, you might try `File > Save and
↳Export Notebook as PDF`, but just make sure you already have `pandoc`
↳installed.

def colab_export_pdf():
    # Modified from: https://medium.com/@jonathanagustin/
    ↳convert-colab-notebook-to-pdf-0ccd8f847dd6
    try:
        import google.colab
        IN_COLAB = True
    except:
        IN_COLAB = False
        print("This cell only works in Google Colab!")
        print("If you are running locally, click File > Export as HTML. Then
↳open the HTML file and save it as a PDF.")

    if IN_COLAB:
        print("Generating PDF. This may take a few seconds.")
        import os, datetime, json, locale, pathlib, urllib, requests, werkzeug,
↳nbformat, google, yaml, warnings
        locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')
        NAME = pathlib.Path(werkzeug.utils.secure_filename(urllib.parse.
↳unquote(requests.get(f"http://{os.environ['COLAB_JUPYTER_IP']}:{os.
↳environ['KMP_TARGET_PORT']}/api/sessions").json()[0]["name"])))
        TEMP = pathlib.Path("/content/pdfs") / f"{datetime.datetime.now().
↳strftime('%Y%m%d_%H%M%S')}_{NAME.stem}"; TEMP.mkdir(parents=True,
↳exist_ok=True)
        NB = [cell for cell in nbformat.reads(json.dumps(google.colab._message.
↳blocking_request("get_ipynb", timeout_sec=30)["ipynb"]), as_version=4).cells
↳if "--Colab2PDF" not in cell.source]
        warnings.filterwarnings('ignore', category=nbformat.validator.
↳MissingIDFieldWarning)
        with (TEMP / f"{NAME.stem}.ipynb").open("w", encoding="utf-8") as
↳nb_copy: nbformat.write(nbformat.v4.new_notebook(cells=NB or [nbformat.v4.
↳new_code_cell("#")]), nb_copy)
        if not pathlib.Path("/usr/local/bin/quarto").exists():
```

```

!wget -q "https://quarto.org/download/latest/quarto-linux-amd64.
↪deb" -P {TEMP} && dpkg -i {TEMP}/quarto-linux-amd64.deb > /dev/null &&↪
↪quarto install tinytex --update-path --quiet
    with (TEMP / "config.yml").open("w", encoding="utf-8") as file: yaml.
↪dump({'include-in-header': [{"text":↪
↪r"\usepackage{fvextra}\DefineVerbatimEnvironment{Highlighting}{Verbatim}{breaksymbolleft={}}
↪ [{"text":↪
↪r"\DefineVerbatimEnvironment{verbatim}{Verbatim}{breaksymbolleft={},showspaces=false,showta
↪file)
    !quarto render {TEMP}/{NAME.stem}.ipynb --metadata-file={TEMP}/config.
↪yaml --to pdf -M latex-auto-install -M margin-top=1in -M margin-bottom=1in -M↪
↪margin-left=1in -M margin-right=1in --quiet
    google.colab.files.download(str(TEMP / f"{NAME.stem}.pdf"))

colab_export_pdf()

```